

Path-Optimal Symbolic Execution of Heap-Manipulating Programs

Pietro Braione
University of Milano-Bicocca
Milano, Italy
pietro.braione@unimib.it

Giovanni Denaro
University of Milano-Bicocca
Milano, Italy
giovanni.denaro@unimib.it

Luca Guglielmo
University of Milano-Bicocca
Milano, Italy
luca.guglielmo@unimib.it

APPENDIX

In this Appendix we introduce an example imperative, object-oriented programming language and give operational semantics for path-optimal symbolic execution. Figure 1 reports the syntax of the language. A program is a sequence of class declarations followed by an expression to be evaluated. Classes have exactly one superclass (there is no multiple inheritance nor interfaces) and multiple fields and methods. The body of a method is a single expression.

There are many types of expressions, but here we want focus on expressions that read the content of object fields (getfield expressions $e.f$), modify the content of object fields (putfield expressions $e.f := e$), conditional expressions $\text{if } e \text{ } e$, and method invocation expressions $e.m(\bar{e})$. These are the expressions whose semantics is more affected by the fact that their subexpressions may be symbolic.

Figure 2 reports the syntax of the configurations and the evaluation contexts used in the operational semantics definition. The small-step operational semantics transition relation $\Rightarrow$ is defined in Figure 3 as a suitable composition of two distinct transition relations, the *computation* transition relation $\rightarrow$, and the (reflexive-transitive closure of the) *refinement* transition relation $\rightsquigarrow$. These relations are formally defined in Figure 4, Figure 5 and Figure 6. The computation transition accounts for the steps that implement a computational step performed to progress the evaluation of the current expression in the program. The refinement transition adds information to the current configuration that is already implicit in it, without progressing the evaluation of the program. The added information may come in the shape of a heap object, or of a value in a field of an object.

In POSE each symbolic reference points to a *symbolic object* in the initial heap. In the beginning of symbolic execution these objects do not actually exist in the initial heap, and are materialized on demand (through a suitable refinement step) when the corresponding symbolic reference is used to access the object. This is the only similarity of POSE with generalized symbolic execution. Differently from the latter, where an input reference is initialized to point to the object it may alias to, in POSE every symbolic reference points to a distinct symbolic object, and the alternative effects of the alias relationship are encoded *inside each object*, by means of if-then-else-values

(ite-values, with shape $\text{ite}(\dots)$).

Let us consider the computation semantics of getfield expressions $e.f$, that read the content of field f from a heap object denoted by expression e . Rule Getfield (c5) considers the case where e evaluates to the location l of a *concrete object*, i.e., of an object created after the start of symbolic execution by means of a `new` expression. In this case, the value is simply read from the object field. Rule Getfield (c1) considers the case where e evaluates to a symbol Y with reference type. The rule may fire only if a symbolic object for Y exists in the heap ($Y \in \text{dom}(H)$), the field f is present in the symbolic object ($f \in \text{dom}(H(Y))$), and the slot corresponding to the field f has been initialized with some value ($H(Y)(f) \neq \perp$). We will discuss all these conditions when explaining the refinement transitions. When all the conditions are met, the value is read from the object field, exactly as for concrete objects. Finally, rules Getfield (c2)-(c4) cope with $\text{ite}(\sigma, \sigma_1, \sigma_2)$ symbolic references. These symbolic references may emerge as expressions of alias conditions and are handled by reading the field f from the objects denoted by the expressions σ_1 and σ_2 , and returning the ite-expression combining the results (rule Getfield (c2)). The resulting path condition is obtained by suitably merging the path conditions obtained by evaluating $\sigma_1.f$ and $\sigma_2.f$, by means of the function $\Sigma' = \text{mergePC}(\sigma, \Sigma'_1, \Sigma'_2)$ defined as follows: If $\Sigma'_1 = \sigma_{11}, \dots, \sigma_{1m}$ and $\Sigma'_2 = \sigma_{21}, \dots, \sigma_{2w}$, then $\Sigma' = (\sigma \Rightarrow \sigma_{11}), \dots, (\sigma \Rightarrow \sigma_{1m}), (\neg \sigma \Rightarrow \sigma_{21}), \dots, (\neg \sigma \Rightarrow \sigma_{2w})$. Similarly, the resulting heap is obtained by merging the updates that the evaluations of $\sigma_1.f$ and $\sigma_2.f$ apply to the heap, by means of the function $H' = \text{mergeHGf}(H'_1, H'_2, f, \sigma)$ defined as follows: $\text{dom}(H') = \text{dom}(H'_1) \cup \text{dom}(H'_2)$, and for all $u \in \text{dom}(H')$ it is:

- 1) if $u \in \text{dom}(H'_1)$ and $u \notin \text{dom}(H'_2)$, then $H'(u) = H'_1(u)$;
- 2) if $u \in \text{dom}(H'_2)$ and $u \notin \text{dom}(H'_1)$, then $H'(u) = H'_2(u)$;
- 3) if $u \in \text{dom}(H'_1)$, and $u \in \text{dom}(H'_2)$, and $H'_1(u) = H'_2(u)$, then $H'(u) = H'_1(u) = H'_2(u)$;
- 4) if $u \in \text{dom}(H'_1)$, and $u \in \text{dom}(H'_2)$, and $H'_1(u) \neq H'_2(u)$, then $\text{dom}(H'(u)) = \text{dom}(H'_1(u)) \cup \text{dom}(H'_2(u))$, and for all $f \in \text{dom}(H'(u))$:

Programs	P	$::=$	$\overline{C} \ e$
Classes	C	$::=$	$\text{class } c : c \{ \overline{t} \ \overline{f}; \overline{D} \}$
Methods	D	$::=$	$t \ m(\overline{t} \ \overline{x}) := e$
Program expressions	e	$::=$	$x \mid v \mid \text{new } c \mid e.f \mid e.f := e \mid \text{let } x := e \text{ in } e$ $\mid e \text{ op } e \mid \text{op } e \mid \text{if } e \ e \ e \mid e.m(\overline{e})$
Types	t	$::=$	$c \mid \text{bool} \mid \text{int}$
Values	v	$::=$	$p \mid u$
Primitive values	p	$::=$	$r \mid X$
Reference values	u	$::=$	$\text{null} \mid l \mid Y$
Literals	r	$::=$	$b \mid n$
Booleans	b	$::=$	$\text{true} \mid \text{false}$
Integers	n	$::=$	$\dots - 1 \mid 0 \mid 1 \dots$
Operators	op	$::=$	$+ \mid = \dots$
Locations	l		
Identifiers	c, f, m, x, X, Y		

Fig. 1. Language syntax

Configurations	J	$::=$	$H \ \Sigma \ \eta$
Configuration expressions	η	$::=$	$x \mid \sigma \mid \text{new } c \mid \eta.f \mid \eta.f := \eta \mid \text{let } x := \eta \text{ in } \eta$ $\mid \eta \text{ op } \eta \mid \text{op } \eta \mid \text{if } \eta \ \eta \ \eta \mid \eta.m(\overline{\eta})$
Heaps	H	$::=$	$\overline{u} \mapsto \overline{o}, \ u \neq \text{null}$
Objects	o	$::=$	$(\overline{f} \mapsto \overline{\sigma})^c \mid (\overline{f} \mapsto \overline{\sigma})$
Path conditions	Σ	$::=$	$\emptyset \mid \Sigma, \sigma$
Symbolic values	σ	$::=$	$\perp \mid v \mid \sigma \text{ op } \sigma \mid \text{op } \sigma \mid Y \leq c \mid Y.f = Z \mid \text{ite}(Y = Y, \sigma, \sigma)$
Evaluation contexts	E	$::=$	$- \mid E.f \mid E.f := E \mid \sigma.f := E \mid \text{let } x := E \text{ in } \eta$ $\mid E \text{ op } \eta \mid \sigma \text{ op } E \mid \text{op } E \mid \text{if } E \ \eta \ \eta \mid E.m(\overline{\eta})$ $\mid \sigma.m(\overline{\sigma}, E, \overline{\eta})$

Fig. 2. Syntax of configurations and evaluation contexts

$$\text{Step:} \frac{H \ \Sigma \ \eta \rightsquigarrow^* H' \ \Sigma' \ \eta \quad H' \ \Sigma' \ \eta \rightarrow H'' \ \Sigma'' \ \eta''}{H \ \Sigma \ \eta \Rightarrow H'' \ \Sigma'' \ \eta''}$$

Fig. 3. Small-step operational semantics

- a) if $f \in \text{dom}(H'_1(u))$ and $f \notin \text{dom}(H'_2(u))$, then $H'(u)(f) = H'_1(u)(f)$;
- b) if $f \in \text{dom}(H'_2(u))$ and $f \notin \text{dom}(H'_1(u))$, then $H'(u)(f) = H'_2(u)(f)$;
- c) if $f \in \text{dom}(H'_1(u))$, and $f \in \text{dom}(H'_2(u))$, and $H'_1(u)(f) = H'_2(u)(f)$ then $H'(u)(f) = H'_1(u)(f) = H'_2(u)(f)$;
- d) if $f \in \text{dom}(H'_1(u))$, and $f \in \text{dom}(H'_2(u))$, and $H'_2(u)(f) = \perp$ then $H'(u)(f) = H'_1(u)(f)$;
- e) if $f \in \text{dom}(H'_1(u))$, and $f \in \text{dom}(H'_2(u))$, and $H'_1(u)(f) = \perp$ then $H'(u)(f) = H'_2(u)(f)$;

- f) if $f \in \text{dom}(H'_1(u))$, and $f \in \text{dom}(H'_2(u))$, and $H'_1(u)(f) \neq H'_2(u)(f)$, and $H'_1(u)(f) \neq \perp$, and $H'_2(u)(f) \neq \perp$ then $H'(u)(f) = \text{ite}(\sigma, H'_1(u)(f), H'_2(u)(f))$.

In the case one of the field access expression $\sigma_1.f$, $\sigma_2.f$ cannot evaluate, the ite-based getfield reduces to the other field access expression (rules Getfield (c3) and (c4)).

Now we analyze the refinement semantics of getfield expressions. Rule Getfield (r1) handles the situation where a symbolic reference has no corresponding symbolic object in the heap yet—i.e., is *unbound*. For the sake of simplicity we

$$\begin{array}{c}
\text{New:} \frac{fields(c) = \bar{f} \quad l \notin \text{dom}(H)}{H \Sigma \text{ new } c \rightarrow H, l \mapsto (\bar{f} \mapsto \text{default})^c \Sigma l} \\
\\
\text{Getfield (c1):} \frac{Y \in \text{dom}(H) \quad f \in \text{dom}(H(Y)) \quad H(Y)(f) \neq \perp}{H \Sigma Y.f \rightarrow H \Sigma H(Y)(f)} \\
\\
\text{Getfield (c2):} \frac{H \Sigma \sigma_1.f \Rightarrow H'_1 \Sigma'_1 \sigma'_1 \quad H \Sigma \sigma_2.f \Rightarrow H'_2 \Sigma'_2 \sigma'_2 \quad H' = \text{mergeHGF}(H'_1, H'_2, f, \sigma) \quad \Sigma' = \text{mergePC}(\sigma, \Sigma'_1, \Sigma'_2)}{H \Sigma \text{ ite}(\sigma, \sigma_1, \sigma_2).f \rightarrow H' \Sigma' \text{ ite}(\sigma, \sigma'_1, \sigma'_2)} \\
\\
\text{Getfield (c3):} \frac{H \Sigma \sigma_1.f \Rightarrow H' \Sigma' \sigma'_1 \quad H \Sigma \sigma_2.f \not\Rightarrow}{H \Sigma \text{ ite}(\sigma, \sigma_1, \sigma_2).f \rightarrow H' \Sigma', \sigma \sigma'_1} \\
\\
\text{Getfield (c4):} \frac{H \Sigma \sigma_1.f \not\Rightarrow \quad H \Sigma \sigma_2.f \Rightarrow H' \Sigma' \sigma'_2}{H \Sigma \text{ ite}(\sigma, \sigma_1, \sigma_2).f \rightarrow H' \Sigma', \neg \sigma \sigma'_2} \\
\\
\text{Getfield (c5):} \frac{}{H \Sigma l.f \rightarrow H \Sigma H(l)(f)} \\
\\
\text{Putfield (c1):} \frac{Y \in \text{dom}(H) \quad f \in \text{dom}(H(Y)) \quad H_r = \text{update}(H, Y, f, \sigma')}{H \Sigma Y.f := \sigma' \rightarrow H_r[Y \mapsto H(Y)[f \mapsto \sigma']] \Sigma \sigma'} \\
\\
\text{Putfield (c2):} \frac{H \Sigma \sigma_1.f := \sigma' \Rightarrow H'_1 \Sigma'_1 \sigma' \quad H \Sigma \sigma_2.f := \sigma' \Rightarrow H'_2 \Sigma'_2 \sigma' \quad H' = \text{mergeHPf}(H'_1, H'_2, f, \sigma) \quad \Sigma' = \text{mergePC}(\sigma, \Sigma'_1, \Sigma'_2), \text{mergeClauses}(H'_1, H'_2, f, \sigma)}{H \Sigma \text{ ite}(\sigma, \sigma_1, \sigma_2).f := \sigma' \rightarrow H' \Sigma' \sigma'} \\
\\
\text{Putfield (c3):} \frac{H \Sigma \sigma_1.f := \sigma' \Rightarrow H' \Sigma' \sigma' \quad H \Sigma \sigma_2.f := \sigma' \not\Rightarrow}{H \Sigma \text{ ite}(\sigma, \sigma_1, \sigma_2).f := \sigma' \rightarrow H' \Sigma', \sigma \sigma'} \\
\\
\text{Putfield (c4):} \frac{H \Sigma \sigma_1.f := \sigma' \not\Rightarrow \quad H \Sigma \sigma_2.f := \sigma' \Rightarrow H' \Sigma' \sigma'}{H \Sigma \text{ ite}(\sigma, \sigma_1, \sigma_2).f := \sigma' \rightarrow H' \Sigma', \neg \sigma \sigma'} \\
\\
\text{Putfield (c5):} \frac{}{H \Sigma l.f := \sigma' \rightarrow H[l \mapsto H(l)[f \mapsto \sigma']] \Sigma \sigma'} \\
\\
\text{Let:} \frac{}{H \Sigma \text{ let } x := \sigma \text{ in } e \rightarrow H \Sigma e[\sigma/x]} \quad \text{Op (c1):} \frac{r_1 \text{ op } r_2 = r}{H \Sigma r_1 \text{ op } r_2 \rightarrow H \Sigma r} \\
\\
\text{Op (c2):} \frac{\text{op } r_1 = r}{H \Sigma \text{ op } r_1 \rightarrow H \Sigma r} \quad \text{Eq (c1):} \frac{}{H \Sigma v = v \rightarrow H \Sigma \text{ true}} \\
\\
\text{Eq (c2):} \frac{r_1 \neq r_2}{H \Sigma r_1 = r_2 \rightarrow H \Sigma \text{ false}} \quad \text{Eq (c3):} \frac{l_1 \neq l_2}{H \Sigma l_1 = l_2 \rightarrow H \Sigma \text{ false}} \\
\\
\text{Eq (c4):} \frac{}{H \Sigma l = \text{null} \rightarrow H \Sigma \text{ false}} \quad \text{Eq (c5):} \frac{}{H \Sigma \text{ null} = l \rightarrow H \Sigma \text{ false}} \\
\\
\text{Eq (c6):} \frac{}{H \Sigma Y = l \rightarrow H \Sigma \text{ false}} \quad \text{Eq (c7):} \frac{}{H \Sigma l = Y \rightarrow H \Sigma \text{ false}}
\end{array}$$

Fig. 4. Small-step operational semantics, computation transition (pt. 1)

$$\begin{array}{c}
\text{If (c1): } \frac{}{H \Sigma \text{ if true } e_1 \ e_2 \rightarrow H \Sigma e_1} \\
\\
\text{If (c2): } \frac{}{H \Sigma \text{ if false } e_1 \ e_2 \rightarrow H \Sigma e_2} \\
\\
\text{If (c3): } \frac{}{H \Sigma \text{ if } \sigma \ e_1 \ e_2 \rightarrow H \Sigma, \sigma \ e_1} \quad \text{If (c4): } \frac{}{H \Sigma \text{ if } \sigma \ e_1 \ e_2 \rightarrow H \Sigma, \neg \sigma \ e_2} \\
\\
\text{Invoke (c1): } \frac{H(l) = o^c \quad \text{recv}(c, m) = c' \quad \text{method}(c', m) = t \ m \ (\overline{t \ x}) \ \{e_m\}}{H \Sigma l.m(\overline{\sigma}) \rightarrow H \Sigma e_m[l/\text{this}][\overline{\sigma/x}]} \\
\\
\text{Invoke (c2): } \frac{\text{method}(c', m) = t \ m \ (\overline{t \ x}) \ \{e_m\} \quad O = \{c \mid \text{overrides}(c, m, c')\}}{H \Sigma Y.m(\overline{\sigma}) \rightarrow H \Sigma, \neg Y = \text{null}, Y \leq: c', \neg Y \leq: \overline{c_{c \in O}} \ e_m[l/\text{this}][\overline{\sigma/x}]} \\
\\
\text{Invoke (c3): } \frac{H \Sigma \sigma_1.m(\overline{\sigma}) \rightarrow H \Sigma'_1 \ \eta'_1}{H \Sigma \text{ ite}(\sigma, \sigma_1, \sigma_2).m(\overline{\sigma}) \rightarrow H \Sigma'_1, \sigma \ \eta'_1} \\
\\
\text{Invoke (c4): } \frac{H \Sigma \sigma_2.m(\overline{\sigma}) \rightarrow H \Sigma'_2 \ \eta'_2}{H \Sigma \text{ ite}(\sigma, \sigma_1, \sigma_2).m(\overline{\sigma}) \rightarrow H \Sigma'_2, \neg \sigma \ \eta'_2} \\
\\
\text{Ctx (c): } \frac{H \Sigma \eta \rightarrow H' \ \Sigma' \ \eta'}{H \Sigma E[\eta/-] \rightarrow H' \ \Sigma' \ E[\eta'/-]}
\end{array}$$

Fig. 5. Small-step operational semantics, computation transition (pt. 2)

$$\begin{array}{c}
\text{Getfield (r1): } \frac{Y \notin \text{dom}(H) \quad \text{fields}(\text{class}(f)) = \overline{f'}}{H \Sigma Y.f \rightsquigarrow H, Y \mapsto (\overline{f'} \mapsto \perp) \ \Sigma, \neg Y = \text{null}, Y \leq: c \ Y.f} \\
\\
\text{Getfield (r1.b): } \frac{Y \in \text{dom}(H) \quad f \notin \text{dom}(H(Y)) \quad \text{fields}(\text{class}(f)) \setminus \text{fields}(H(Y)) = \overline{f'}}{H \Sigma Y.f \rightsquigarrow H[Y \mapsto (H(Y), \overline{f'} \mapsto \perp)] \ \Sigma, Y \leq: c \ Y.f} \\
\\
\text{Getfield (r2): } \frac{Y \in \text{dom}(H) \quad f \in \text{dom}(H(Y)) \quad H(Y)(f) = \perp \quad Z \text{ fresh}}{H \Sigma Y.f \rightsquigarrow H[Y \mapsto H(Y)[f \mapsto \text{assume}(H, Y, f, Z)]] \ \Sigma, Y.f = Z \ Y.f} \\
\\
\text{Putfield (r1): } \frac{Y \notin \text{dom}(H) \quad \text{fields}(\text{class}(f)) = \overline{f'}}{H \Sigma Y.f := \sigma' \rightsquigarrow H, Y \mapsto (\overline{f'} \mapsto \perp) \ \Sigma, \neg Y = \text{null}, Y \leq: c \ Y.f := \sigma'} \\
\\
\text{Putfield (r1.b): } \frac{Y \in \text{dom}(H) \quad f \notin \text{dom}(H(Y)) \quad \text{fields}(\text{class}(f)) \setminus \text{fields}(H(Y)) = \overline{f'}}{H \Sigma Y.f := \sigma' \rightsquigarrow H[Y \mapsto (H(Y), \overline{f'} \mapsto \perp)] \ \Sigma, Y \leq: c \ Y.f := \sigma'} \\
\\
\text{Ctx (r): } \frac{H \Sigma \eta \rightsquigarrow H' \ \Sigma' \ \eta'}{H \Sigma E[\eta/-] \rightsquigarrow H' \ \Sigma' \ E[\eta'/-]}
\end{array}$$

Fig. 6. Small-step operational semantics, refinement transition

suppose that each field f is contained in exactly one class. If $c = \text{class}(f)$ is the class that contains the field f , then the refinement step adds a symbolic object of class c to the heap, whose fields are all uninitialized (the function $\text{fields}(c)$ yields the set of all the fields declared in c and in its superclasses), binds Y to the new object, and updates the path condition to reflect the fact that field access has success ($\neg Y = \text{null}$) and that the symbolic object's type is a subclass of c ($Y \leq c$). Rule Getfield (r1.b) handles the situation where Y has an associated symbolic object in the heap, but this does not contain the field f . If we assume the programs to be type-correct, this situation may only arise if the symbolic object was previously created with the help of a superclass of c . Note that, upon creation, a symbolic object is assumed to possibly be of *any subclass* of its creation class. If later during symbolic execution a finer type assumption must be performed, as in this case, the missing fields are added to the object, with all the new slots left uninitialized. Also, the path condition is updated to reflect the new type assumption on the symbolic object bound to Y ($Y \leq c$). Finally, the rule Getfield (r2) handles the situation where the field f in the symbolic object pointed by Y is uninitialized ($H(Y)(f) = \perp$). The refinement rule therefore initializes the field with a special value $\text{assume}(H, Y, f, Z)$ that accounts for all the possible alias relations between the symbolic reference Y and all the other symbolic references that are bound. Formally the function $\text{assume}(H, Y, f, Z)$ is defined as follows:

$$\text{assume}(H, Y, f, Z) = \text{ite}(Y = Y_1, H(Y_1)(f), \dots, \text{ite}(Y = Y_m, H(Y_m)(f), Z) \dots)$$

where $\{Y_1 \dots Y_m\} = \{Y' \in \text{dom}(H) \mid Y' \neq Y \wedge f \in \text{dom}(H(Y')) \wedge H(Y')(f) \neq \perp\}$. The set $\{Y_1 \dots Y_m\}$ is the *alias set* of Y , and is composed by the set of all the bound symbolic references that point to a type-compatible object with c , and whose f field is initialized. The symbolic value Z stands for the initial value of $Y.f$, the value that would have the field $Y.f$ if symbolic execution up to the current instant of time did not modify the field. The *assume* value that is assigned to the $Y.f$ field reflects the possible current value of the field: For all Y_i , If Y aliases Y_i , then $Y.f$ has the value stored in $Y_i.f$. If Y does not alias any object in its alias set, then it still contains the unmodified initial value Z .

Figure 7 reports an example illustrating the effect of the evaluation of a field reading expression. Let us suppose that the field $Y_1.f$ is uninitialized, that the expression $Y_1.f$ must be evaluated, and that two symbolic references Y_2 and Y_3 are bound in the heap with a value for field f . The effect of refinement is to initialize field $Y_1.f$ to the *ite* value shown in figure, which takes into account the possible alias relations of Y_1 with Y_2 and Y_3 . This value is returned as a result of the evaluation of the expression.

The computation semantics of field assignment expressions, $e.f := e$, can be described as follows. Rule Putfield (c5) is for concrete objects: If l is the location of a concrete object, the field f of the object is directly updated. More complex is the

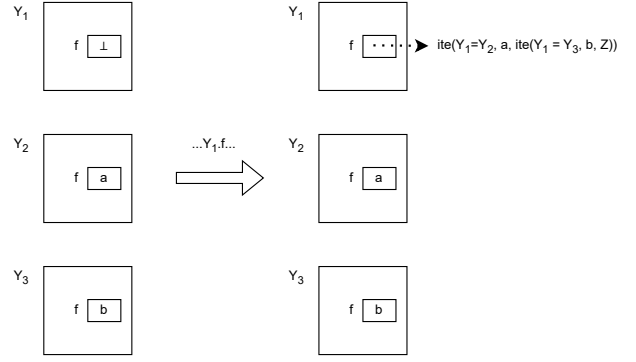


Fig. 7. Example semantics of field reading expressions

effect of updating a field of a symbolic object, as described by rule Putfield (c1). The update can be performed only if Y is bound and the symbolic object has field f : Otherwise, refinement is necessary: The corresponding refinement transitions, Putfield (r1) and Putfield (r1.b), are almost identical to Getfield (r1) and Getfield (r1.b), and will not be discussed. The effect of a field update does not only affect the symbolic object bound to Y : It affects all the symbolic objects in the alias set of Y . Formally the impact of the update on the alias set is described by the definition of $H_r = \text{update}(H, Y, f, \sigma')$:

$$\begin{aligned} \text{dom}(H_r) &= \text{dom}(H) \wedge \forall u' \in \text{dom}(H), f' \in \text{dom}(H)(Y') \mid \\ &u' = Y' \wedge Y' \neq Y \wedge f' = f \wedge H(Y')(f) \neq \perp \\ &\implies H_r(u')(f') = \text{ite}(Y = Y', \sigma', H(u')(f')) \wedge \\ &\neg(u' = Y' \wedge Y' \neq Y \wedge f' = f \wedge H(Y')(f) \neq \perp) \\ &\implies H_r(u')(f') = H(u')(f'). \end{aligned}$$

In plain terms, if Y' is a bound symbolic reference to a type-compatible object, whose f field is initialized, the field $Y'.f$ must also be updated to an *ite* reflecting the potential alias relation: If Y aliases Y' , then $Y'.f$ also assumes the value assigned to $Y.f$, otherwise it retains its current value. Finally, rules Putfield (c2)-(c4) handle the case where an *ite* ($\sigma, \sigma_1, \sigma_2$) symbolic reference is used to refer the object to be updated. Rules Putfield (c3) and (c4) are analogous to Getfield (c3) and (c4), and will not be discussed. Rule Putfield (c2) defines the effects of a field update on the heap and the path condition in terms of a suitable merging between the effects of updating the objects referred by σ_1 and σ_2 respectively. The formal definitions of the *mergeHPf* and *mergeClauses* functions are as follows. It is $H' = \text{mergeHPf}(H'_1, H'_2, f, \sigma)$ iff $\text{dom}(H') = \text{dom}(H'_1) \cup \text{dom}(H'_2)$, and for all $u \in \text{dom}(H')$ it is:

- 1) if $u \in \text{dom}(H'_1)$, and $u \notin \text{dom}(H'_2)$, then $\text{dom}(H'(u)) = \text{dom}(H'_1(u))$, and for all $f \in \text{dom}(H'(u))$:
 - a) if $H'_1(u)(f) = \perp$ then it is $H'(u)(f) = \perp$;
 - b) otherwise it is $H'(u)(f) = \text{ite}(\sigma, \sigma_1, Z)$, where $\sigma_1 = H'_1(u)(f)$ and Z is a fresh symbol;
- 2) if $u \notin \text{dom}(H'_1)$, and $u \in \text{dom}(H'_2)$, then $\text{dom}(H'(u)) = \text{dom}(H'_2(u))$, and for all $f \in \text{dom}(H'(u))$:

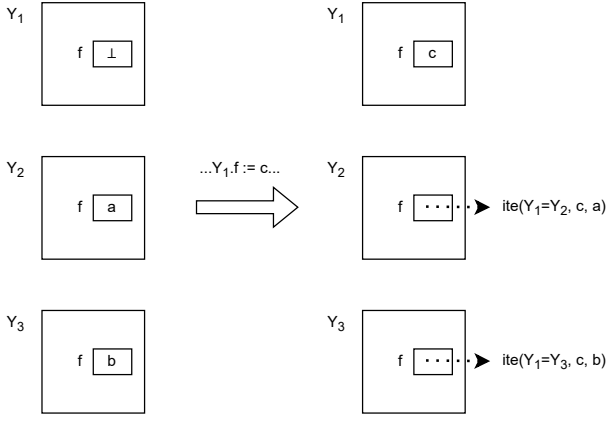


Fig. 8. Example semantics of field assignment expressions

- a) if $H'_2(u)(f) = \perp$ then it is $H'(u)(f) = \perp$;
- b) otherwise it is $H'(u)(f) = \text{ite}(\sigma, Z, \sigma_2)$, where $\sigma_2 = H'_2(u)(f)$ and Z is a fresh symbol;
- 3) if $u \in \text{dom}(H'_1)$, and $u \in \text{dom}(H'_2)$, and $H'_1(u) = H'_2(u)$, then $H'(u) = H'_1(u) = H'_2(u)$;
- 4) if $u \in \text{dom}(H'_1)$, and $u \in \text{dom}(H'_2)$, and $H'_1(u) \neq H'_2(u)$, then $\text{dom}(H'(u)) = \text{dom}(H'_1(u)) \cup \text{dom}(H'_2(u))$, and for all $f \in \text{dom}(H'(u))$:
 - a) if $f \in \text{dom}(H'_1(u))$ and $f \notin \text{dom}(H'_2(u))$, then:
 - i) if $H'_1(u)(f) = \perp$ then $H'(u)(f) = \perp$;
 - ii) otherwise, $H'(u)(f) = \text{ite}(\sigma, \sigma_1, Z)$, where $\sigma_1 = H'_1(u)(f)$ and Z is a fresh symbol;
 - b) if $f \in \text{dom}(H'_2(u))$ and $f \notin \text{dom}(H'_1(u))$, then:
 - i) if $H'_2(u)(f) = \perp$ then $H'(u)(f) = \perp$;
 - ii) otherwise, $H'(u)(f) = \text{ite}(\sigma, Z, \sigma_2)$, where $\sigma_2 = H'_2(u)(f)$ and Z is a fresh symbol;
 - c) if $f \in \text{dom}(H'_1(u))$, and $f \in \text{dom}(H'_2(u))$, and $H'_1(u)(f) = H'_2(u)(f)$ then $H'(u)(f) = H'_1(u)(f) = H'_2(u)(f)$;
 - d) if $f \in \text{dom}(H'_1(u))$, and $f \in \text{dom}(H'_2(u))$, and $H'_1(u)(f) \neq \perp$, and $H'_2(u)(f) = \perp$, then $H'(u)(f) = \text{ite}(\sigma, \sigma_1, Z)$, where $\sigma_1 = H'_1(u)(f)$ and Z is a fresh symbol;
 - e) if $f \in \text{dom}(H'_1(u))$, and $f \in \text{dom}(H'_2(u))$, and $H'_1(u)(f) = \perp$, and $H'_2(u)(f) \neq \perp$, then $H'(u)(f) = \text{ite}(\sigma, Z, \sigma_2)$, where $\sigma_2 = H'_2(u)(f)$ and Z is a fresh symbol;
 - f) if $f \in \text{dom}(H'_1(u))$, and $f \in \text{dom}(H'_2(u))$, and $H'_1(u)(f) \neq H'_2(u)(f)$, and $H'_1(u)(f) \neq \perp$, and $H'_2(u)(f) \neq \perp$ then $H'(u)(f) = \text{ite}(\sigma, \sigma_1, \sigma_2)$, where $\sigma_1 = H'_1(u)(f)$ and $\sigma_2 = H'_2(u)(f)$.

The function $\text{mergeClauses}(H'_1, H'_2, f, \sigma)$ yields the set of all the clauses $Y.f = Z$, where Z are all (and only) the fresh symbols introduced in the definition of mergeHPf , and Y is the symbolic input reference that points to the symbolic object where Z has been injected.

Figure 8 reports an example of the effect of the evaluation of a field assignment expression. We again suppose that three

symbolic references, Y_1, Y_2, Y_3 , are bound to symbolic objects, that all the symbolic objects define the field f , that the object Y_2 and Y_3 have field f initialized, and that the expression $Y_1.f := c$ must be evaluated. In this situation no refinement transition fires: The computation transition updates the field f of Y_1 to the c value, and as a further effect updates the fields f of all the other symbolic objects with ite terms that reflect the possible effect of the update on these objects in case an alias relation subsists.

Conditional expressions $\text{if } e \text{ } e$ have associated only computation transitions. Rules If (c1) and If (c2) manage the cases $\text{if true} \dots$ and $\text{if false} \dots$, that reduce to the evaluation of the corresponding branch. Rules If (c3) and If (c4) manage the case where the condition of the if expression is symbolic. In this case both transitions may fire, and the path condition is updated to reflect the assumption associated to the branch taken.

Finally, method invocation expressions $e.m(\bar{e})$ are handled only by computation transitions. We introduce the following definitions:

- The function $\text{recv}(c, m)$ yields the class that provides the implementation of method m to class c (it is c itself iff c implements m , or an ancestor of c iff c inherits the implementation of m).
- The predicate $\text{impl}(c, m)$ is true iff c implements m .
- The predicate $\text{sees}(c, m, c')$ is true iff c is a subclass of c' , c' implements the method m , and no superclass of c and subclass of c' implements m (note that c may implement m).
- The predicate $\text{overrides}(c, m, c')$ is defined as $c \neq c' \wedge \text{sees}(c, m, c') \wedge \text{impl}(c, m)$.

We can deduce that $\text{recv}(c, m) = c'$, for $c \neq c'$, iff $\text{sees}(c, m, c') \wedge \neg \text{impl}(c, m)$, and that $\text{recv}(c, m) = c$ iff $\text{impl}(c, m)$. Given the above definitions it is easy to see that rule Invoke (c1) (method invocation on concrete objects) finds the implementation of the method of the object pointed by location l , and invokes it. Different is the case (rule Invoke (c2)) where the receiver of the method invocation is a symbolic reference. In this case, since the symbolic reference might refer to an object of any class that declares the method m , symbolic execution branches to all possible implementations of m . The path condition at each branch is assumed to reflect the assumption on the class of the symbolic receiver object: When the method m is executed, if m is declared by class c' , the object may have any subclass of c' that does not override m . The final rules Invoke (c3) and Invoke (c4) handle the case where the reference to the receiver object is an $\text{ite}(\sigma, \sigma_1, \sigma_2)$ symbolic reference. In synthesis, when such a reference is encountered, symbolic execution branches.

We conclude this Appendix by briefly discussing the remaining rules. The Let computation rule has the effect of performing a term substitution in its main argument. The Op (c1) and Op (c2) computation rules yield the result of the application of a binary or unary operator to concrete operands. The Eq (c1)-(c7) computation rules account for the semantics of equality, where syntactically identical values are

always considered equal, distinct concrete locations are always considered different each other and from `null`, and symbolic references are always considered different from concrete locations. Finally, the context rules `Ctx (c)` and `Ctx (r)` extend the reduction semantics of expressions under the admissible evaluation contexts.